

Kubernetes StatefulSets: A Beginner's Guide to Databases

Deployments are great for web apps, but databases need something more stable that won't lose data. StatefulSets give each of your database pods a permanent name and its own dedicated hard drive so your data stays safe.

What you'll learn:

1. Giving Pods Permanent Names
2. Setting Up a Headless Service
3. Attaching Dedicated Storage
4. Scaling Safely One by One

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Giving Pods Permanent Names

Unlike Deployments which create pods with random names like 'web-abc12', StatefulSets name pods starting from zero, like 'db-0' and 'db-1'. This predictable naming is crucial because databases need to know exactly who is the master and who is the replica. If db-0 crashes, Kubernetes recreates it with the exact same name, 'db-0', so the rest of your system doesn't get confused.

```
$kubectl get pods -l app=postgres
```

Cloud & DevOps Daily

Kubernetes StatefulSets: When Deployments Aren't Enough

Sunday, September 27, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Setting Up a Headless Service

Standard Kubernetes services act as load balancers, routing traffic randomly. Databases need direct communication between individual pods to sync data. A headless service does this by setting 'clusterIP: None' in the YAML configuration, which returns the direct IP addresses of all the individual pods instead of a single load balancer IP.

```
$kubectl get svc postgres-headless -o yaml
```

Cloud & DevOps Daily

Kubernetes StatefulSets: When Deployments Aren't Enough

Sunday, September 27, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Attaching Dedicated Storage

Each pod in a StatefulSet needs its own private disk space, called a Volume. By using a 'volumeClaimTemplates' section in your YAML, Kubernetes automatically creates a new, dedicated disk for every single pod you spin up. Even if a pod dies and gets replaced, the new pod automatically connects back to its original disk.

```
$kubectl get pvc -l app=postgres
```

Cloud & DevOps Daily

Kubernetes StatefulSets: When Deployments Aren't Enough

Sunday, September 27, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Scaling Safely One by One

Deployments scale up and down all at once, which can corrupt databases. StatefulSets scale in a strict, orderly fashion: they start 'db-0' first, wait for it to be fully ready, then start 'db-1'. When scaling down, they delete the highest-numbered pod first ('db-1') to make sure data is safely saved before shutting down.

```
$kubectl scale statefulset postgres --replicas=3
```

Cloud & DevOps Daily

Kubernetes StatefulSets: When Deployments Aren't Enough

Sunday, September 27, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always use a Headless Service with your StatefulSets so pods can find each other.
- + Keep your storage class settings dynamic so disks provision automatically.
- + Name your pods clearly so you can easily identify the primary database node.
- + Test your pod deletion to prove that the data persists on the volume.
- + Use readiness probes to ensure pods are fully ready before the next one starts.

Watch Out For

- ! Deleting a StatefulSet does not delete its storage; you must clean up PVCs manually to avoid extra cloud costs.
- ! Forgetting to set 'clusterIP: None' on your database service, which breaks peer-to-peer sync.
- ! Using StatefulSets for stateless apps like frontend webs, which slows down scaling unnecessarily.
- ! Not configuring a storage class, causing pods to get stuck in 'Pending' forever.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh